SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 2 – Agile Sprint Planning & User Story Workshop

Course Code:	CSA10 / CSA1021	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 2 – **Agile Sprint Planning & User Story Workshop**
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:	P. Siddardha	**Reg. No.:**	192424271
Date:	14/10/26	**Duration:**	60 minutes

Code of Conduct

I __p. siddardha(192424271)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ____P. Siddardha_____

Annexure A – Agile Sprint Planning & User Story Workshop

Instructions to Students:

Each student is assigned an individual software project problem statement. Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

1. **Identify the Project Vision and Stakeholders**
   - Define the project's vision, objectives, and identify all key stakeholders involved.
2. **Create Epics and User Stories**
   - Break down the requirements into Epics and formulate well-structured User Stories using the format: *As a <user>, I want <feature>, so that <benefit>.*
3. **Define Acceptance Criteria**
   - Write clear and measurable acceptance criteria for each selected user story using the Given–When–Then format or equivalent.
4. **Prioritize the Product Backlog**
   - Organize and prioritize the product backlog based on business value, customer needs, and project dependencies.
5. **Plan the Sprint Backlog**
   - Select high-priority user stories for the first sprint and divide them into actionable development tasks.
6. **Estimate Effort Using Story Points**
   - Assign story points to each selected user story using an appropriate estimation technique (e.g., Fibonacci sequence or Planning Poker) and justify your estimates.
7. **Present the Sprint Goal and Expected Deliverables**
   - Define a clear sprint goal and describe the expected sprint deliverables, including the features to be completed at the end of the sprint.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Project Vision & Stakeholder Identification (30%)	Project vision is clear, comprehensive, and aligned with the problem statement. All relevant stakeholders and their roles are correctly identified.	Vision is clear with minor omissions. Most stakeholders are correctly identified.	Vision is partially defined. Some stakeholders are missing or incorrectly identified.	Vision is unclear or incomplete. Stakeholders are poorly identified or missing.
Epics & User Stories (25%)	Epics are well-defined and user stories are complete, follow Agile format, and fully capture functional requirements.	Epics and user stories are mostly complete with minor formatting or requirement issues.	User stories are partially complete with limited Agile structure.	User stories are incomplete, unclear, or do not follow Agile practices.
Acceptance Criteria (20%)	Acceptance criteria are specific, measurable, testable, and aligned with all user stories.	Acceptance criteria are mostly clear and testable with minor gaps.	Acceptance criteria are partially defined and lack clarity or completeness.	Acceptance criteria are missing or not aligned with user stories.
Product Backlog Prioritization & Sprint Planning (15%)	Product backlog is logically prioritized, sprint backlog is realistic, and task breakdown is complete.	Backlog prioritization and sprint planning are mostly appropriate with minor issues.	Prioritization or sprint planning is partially complete with limited justification.	Backlog prioritization and sprint planning are poorly organized or incomplete.
Story Point Estimation, Sprint Goal & Presentation (10%)	Story point estimation is well justified, sprint goal is clear, deliverables are realistic, and presentation is professional and well organized.	Estimation and sprint goal are mostly appropriate with minor inconsistencies. Presentation is clear.	Estimation or sprint goal lacks justification. Presentation is adequate but incomplete.	Estimation is unrealistic or missing. Sprint goal, deliverables, and presentation are unclear or incomplete.

Criteria	**Mark**
Project Vision & Stakeholder Identification **(30%)**	/5
Epics & User Stories **(25%)**	/5
Acceptance Criteria **(20%)**	/5
Product Backlog Prioritization & Sprint Planning **(15%)**	/5
Story Point Estimation, Sprint Goal & Presentation **(10%)**	/5
TOTAL	/25

1. PROJECT VISION AND STAKEHOLDERS

1.1 Project Vision

The vision of the LEO Satellite Key Management Fabric is to develop a secure, modular software system that simulates the management of cryptographic keys across a network of Low-Earth Orbit (LEO) satellites and ground control stations. The system aims to automate key generation, verify satellite identities, coordinate encrypted key distribution during short communication windows, support scheduled key rotation, and isolate simulated satellite nodes when suspicious activity is detected.

LEO satellites move rapidly relative to ground stations, resulting in limited communication periods. If a key update cannot be completed before a contact window closes, the system must record the update status and support a controlled retry during a later simulated contact window. Similarly, if a satellite is suspected of compromise, its access must be restricted according to predefined security rules, and the event must be recorded for investigation.

The proposed project will be developed as a software-only academic prototype. Satellite nodes, communication windows, and selected security events will be simulated without requiring physical satellite hardware. A web dashboard will provide an interface for authorized operators to monitor the simulated fleet and manage key-related operations.

The primary vision is to demonstrate how Agile software engineering practices can be used to develop, test, and improve a reliable key management system through small, prioritized, and verifiable increments.

1.2 Project Objectives

The main objectives are:

1. To simulate a fleet of LEO satellite nodes and maintain their identities and operational status.
2. To generate cryptographic keys using an appropriate cryptographic library and manage key metadata securely.
3. To verify satellite-node identities before allowing authorized key management operations.
4. To simulate encrypted key distribution within short satellite communication windows.
5. To support controlled key rotation and maintain the status of each key lifecycle operation.
6. To detect simulated suspicious activity and restrict access to potentially compromised nodes.
7. To maintain tamper-evident audit records of important security and key management events.
8. To provide a web dashboard for viewing satellite status, key operation results, alerts, and audit history.
9. To test functional correctness, access control, interrupted communication handling, and error recovery.
10. To develop the system incrementally using Agile Scrum, with regular reviews, testing, feedback, and controlled software releases.

1.3 Project Scope

In Scope

- Simulated satellite and ground control node management.
- Satellite registration and identity verification.
- Cryptographic key generation and lifecycle metadata management.
- Simulated encrypted key distribution and contact-window handling.
- Key rotation and update-status tracking.
- Simulated compromised-node detection and access isolation.
- Tamper-evident audit logging.
- Role-based access to management functions.
- A web-based monitoring dashboard.

- Automated testing, documentation, version control, and controlled software releases.

Out of Scope

- Connection to actual satellites or operational military networks.
- Transmission of cryptographic keys to real spacecraft.
- Integration with classified systems or real orbital communication infrastructure.
- Certification for operational defense deployment.
- A guarantee of absolute security or complete prevention of cyberattacks.

The prototype will demonstrate and evaluate selected security mechanisms in a controlled environment. Its results will not, by themselves, establish suitability for real-world satellite operations.

1.4 Key Stakeholders and Their Roles

1. Ground Control Operator

Responsible for registering simulated satellites, initiating authorized key operations, monitoring update status, and reviewing alerts.

2. Satellite and Network Operations Team

Provides operational requirements concerning simulated satellite availability, communication windows, failed transmissions, retry handling, and node status.

3. Cybersecurity Team

Defines security requirements, reviews access-control policies, evaluates suspicious-node isolation, and checks whether security events are recorded correctly.

4. Cryptographic Engineer

Advises on appropriate cryptographic algorithms, key generation, key lifecycle management, protected storage, and safe handling of sensitive key material.

5. Software Development Team

Designs and develops the application modules, backend APIs, dashboard, database interactions, and satellite simulation services.

6. Software Testing and Quality Assurance Team

Prepares test cases, verifies acceptance criteria, tests failure scenarios, identifies defects, and confirms that completed features meet the Definition of Done.

7. Database and System Administrator

Manages database configuration, access permissions, backups, environment configuration, and recovery procedures.

8. Security Auditor

Reviews audit records to trace key management actions, authentication events, failed updates, and node-isolation decisions.

9. Project Manager / Scrum Master

Facilitates sprint planning, tracks progress, removes development blockers, coordinates reviews, and supports continuous improvement.

10. Product Owner / Faculty Supervisor

Represents project requirements, prioritizes the product backlog according to value and risk, reviews sprint deliverables, and provides feedback.

11. Authorized System Administrator

Manages system-level configuration, user permissions, and approved maintenance activities.

These stakeholders contribute different perspectives. For example, the ground control operator needs a clear update-status display, while the cybersecurity team needs reliable access controls and traceable audit records. Agile reviews will help ensure that both operational usability and security requirements are addressed.

2. CREATE EPICS AND USER STORIES

An **epic** represents a broad area of system functionality that can be divided into smaller user stories. Each user story describes a specific user need and its expected benefit.
The following epics organize the project requirements into manageable development increments.

Epic 1: Satellite Fleet and Identity Management
User Story 1 – Register a Satellite Node
As a ground control operator, I want to register a simulated satellite node with a unique identifier, so that the system can manage and monitor each satellite separately.
User Story 2 – Verify Satellite Identity
As a cybersecurity administrator, I want the system to verify a satellite node's identity before allowing sensitive operations, so that unauthorized nodes cannot perform key management actions.

Epic 2: Cryptographic Key Management
User Story 3 – Generate Cryptographic Keys
As an authorized key management operator, I want to generate cryptographic keys using an approved cryptographic library, so that authorized simulated satellite nodes can be provisioned with key material securely.
User Story 4 – Track Key Lifecycle and Rotation
As a key management operator, I want to view key status and initiate controlled key rotation, so that keys can be replaced according to the defined lifecycle policy.

Epic 3: Secure Key Distribution and Contact-Window Management
User Story 5 – Distribute Keys During a Contact Window
As a ground control operator, I want to initiate a simulated encrypted key distribution operation during an available contact window, so that the system can demonstrate key updates under limited communication time.
User Story 6 – Recover from an Interrupted Key Update
As a satellite operations engineer, I want the system to record incomplete key updates and retry them during a later valid contact window, so that a temporary communication interruption does not leave the operation's status unclear.

Epic 4: Compromised-Node Detection and Isolation
User Story 7 – Isolate a Suspicious Satellite Node
As a cybersecurity administrator, I want the system to restrict a simulated satellite node when a configured compromise condition is detected, so that the node cannot continue unauthorized key management operations.

Epic 5: Security Auditing and Monitoring
User Story 8 – Maintain Audit Records
As a security auditor, I want important key management and security events to be recorded in tamper-evident audit logs, so that I can trace actions and investigate suspicious activity.
User Story 9 – Monitor the Satellite Fleet
As a ground control operator, I want a dashboard showing satellite status, key operation status, communication-window results, and security alerts, so that I can monitor the simulated system from one interface.

Epic 6: System Administration and Access Control
User Story 10 – Manage Authorized User Access
As a system administrator, I want to assign roles and permissions to system users, so that each user can access only the functions permitted by their responsibilities.
These epics cover the main functional requirements while keeping related features grouped together. Dependencies are also considered: satellite registration supports identity verification, identity verification supports protected key operations, and reliable audit records support investigation and accountability.

3. DEFINE ACCEPTANCE CRITERIA

Acceptance criteria specify the conditions that must be satisfied before a user story can be accepted. The Given–When–Then format makes the expected behavior clear and testable.

User Story 1: Register a Satellite Node
Given an authorized operator is logged in,
When the operator submits a valid, unique satellite identifier and the required node details,
Then the system must register the node, assign its initial status, and display a successful confirmation.
Additional acceptance criteria:

- The system must reject a duplicate satellite identifier.
- Required fields must be validated before registration.
- Invalid submissions must display a meaningful error message.
- The registered node must appear in the satellite fleet list.

User Story 2: Verify Satellite Identity
Given a satellite node has been registered,
When the node requests an operation that requires identity verification,
Then the system must validate the configured identity credentials before authorizing the operation.
Additional acceptance criteria:

- A valid identity must pass verification.
- Invalid or missing credentials must cause the operation to be rejected.
- Rejected authentication attempts must be recorded in the audit log.
- A failed verification must not result in key material being disclosed.

User Story 3: Generate Cryptographic Keys
Given an authorized user selects an eligible registered satellite node,
When the user requests key generation,
Then the system must generate key material using the configured cryptographic library and record the operation's status and associated metadata.
Additional acceptance criteria:

- The system must reject key-generation requests from unauthorized users.
- The key operation must be associated with the intended node and a unique key identifier.
- Raw key material must not be displayed in the dashboard or stored as ordinary plaintext in the database.
- The result of the operation must be recorded in the audit log without including the secret key itself.

User Story 4: Track Key Lifecycle and Rotation
Given an eligible satellite node has an existing managed key,
When an authorized operator initiates a permitted rotation operation,
Then the system must create a rotation record and update the key lifecycle status according to the configured workflow.
Additional acceptance criteria:

- The system must identify the key and node associated with the rotation.
- The operation must record its start time, status, and completion or failure result.
- A failed rotation must not be reported as successful.
- The previous key must not be removed prematurely if the configured workflow requires confirmation of the replacement key.

User Story 5: Distribute Keys During a Contact Window
Given a registered node has passed identity verification and a simulated contact window is available,
When an authorized operator starts a key distribution operation,
Then the simulator must attempt the configured encrypted transfer and record whether the operation completed within the available window.
Additional acceptance criteria:

- The operation must be rejected if the node is unauthorized or isolated.
- The simulator must identify whether a contact window is open or closed.
- A completed transfer must be recorded as successful only after the configured completion condition is satisfied.
- A failed or expired transfer must be recorded with an appropriate status.

User Story 6: Recover from an Interrupted Key Update
Given a simulated key distribution operation is interrupted because the contact window closes,
When the system processes the interruption,
Then it must record the operation as incomplete or failed according to the defined workflow and preserve enough state to support a controlled retry.
Additional acceptance criteria:

- The system must not mark an incomplete update as successful.
- The interruption reason and timestamp must be recorded.
- A retry must be permitted only when the configured conditions are satisfied.
- Repeated requests must not accidentally create conflicting key lifecycle states.

User Story 7: Isolate a Suspicious Satellite Node
Given a simulated satellite node triggers a configured compromise condition,
When the security service evaluates the event and applies the isolation policy,
Then the system must change the node's status to isolated and block further protected key management operations from that node.
Additional acceptance criteria:

- The isolation event must be recorded with a timestamp and reason.
- The dashboard must display the isolated status.
- An isolated node must not perform restricted operations until an authorized recovery procedure permits it.
- The system must distinguish a simulated detection event from a confirmed real-world compromise.

User Story 8: Maintain Audit Records
Given an important security or key management operation occurs,
When the operation is processed,
Then the system must record the relevant event details in an audit log using a tamper-evident record structure.
Additional acceptance criteria:

- Each record must include an event identifier, timestamp, event type, and relevant non-secret metadata.
- The log must not contain raw cryptographic key material or authentication secrets.
- The integrity-verification process must detect a modification to a previously recorded hash-chained entry.
- Authorized auditors must be able to retrieve relevant records for investigation.

User Story 9: Monitor the Satellite Fleet
Given satellite and key operation records are available,
When an authorized operator opens the dashboard,
Then the system must display the current recorded node statuses, recent key operation results, and available security alerts.
Additional acceptance criteria:

- Isolated nodes must be visually distinguishable from active nodes.
- Failed or incomplete key operations must be identifiable.
- The dashboard must show a meaningful message when no records are available.
- Users without the required permission must not access restricted information.

User Story 10: Manage Authorized User Access
Given an authorized administrator is managing system accounts,
When the administrator assigns a supported role or permission,
Then the system must enforce that permission on the relevant protected operations.
Additional acceptance criteria:

- Users must be denied access to operations outside their assigned permissions.
- Invalid or unauthorized permission changes must be rejected.
- Access-control changes must be recorded in the audit log.
- The interface must not be the only enforcement point; the backend must also validate authorization.

The acceptance criteria will be used to prepare test cases during each sprint. A user story will not be considered complete merely because its interface exists; its expected behavior must also be verified.

4. PRIORITIZE THE PRODUCT BACKLOG

The product backlog contains all identified user stories, ordered according to security importance, business value, dependencies, and their contribution to a usable prototype.
The **MoSCoW prioritization technique** is used to classify requirements as Must Have, Should Have, Could Have, or Won't Have for the current release. Priority is distinct from story points: priority indicates what should be delivered first, while story points estimate relative complexity and uncertainty.

4.1 Product Backlog

Priority 1 – Must Have

1. **User Story 1: Register a Satellite Node.** Provides the foundation for identifying and managing simulated nodes.

2. **User Story 2: Verify Satellite Identity.** Establishes access control before sensitive operations.
3. **User Story 3: Generate Cryptographic Keys.** Demonstrates the central key management function.
4. **User Story 8: Maintain Audit Records.** Supports traceability of key and security operations.

Priority 2 – Must Have for the Core Distribution Workflow

5. **User Story 5: Distribute Keys During a Contact Window.** Demonstrates the project's key distribution requirement.
6. **User Story 6: Recover from an Interrupted Key Update.** Handles a defining LEO communication challenge.

Priority 3 – Should Have

7. **User Story 4: Track Key Lifecycle and Rotation.** Supports controlled key replacement and lifecycle management.
8. **User Story 10: Manage Authorized User Access.** Extends role and permission management beyond the initial access-control baseline.

Priority 4 – Should Have

9. **User Story 7: Isolate a Suspicious Satellite Node.** Adds a simulated response to suspicious activity and reduces the risk of unauthorized operations within the prototype.

Priority 5 – Could Have for the Initial Release

10. **User Story 9: Monitor the Satellite Fleet.** Provides a consolidated dashboard for operational visibility. Basic status information may be shown during early development, with richer monitoring added after core services are available.

Won't Have in the Current Academic Prototype

- Integration with real satellites or classified networks.
- Operational deployment in an actual defense environment.
- Claims of absolute security or guaranteed prevention of all data leakage.

These exclusions keep the project feasible within an academic development cycle. They do not imply that monitoring or isolation is unimportant; their detailed implementation can follow the core security and distribution workflow.

4.2 Prioritization Rationale and Dependencies

The backlog follows the dependency chain below:

Satellite registration → Identity verification → Key generation → Key distribution → Key rotation and recovery

Audit logging should be incorporated into the relevant operations from the beginning rather than postponed until the end. This helps ensure that important security actions are traceable during development.

The first sprint will establish the foundation for the key management workflow. Contact-window simulation and interrupted-transfer recovery will follow once the necessary node and key services are available. Dashboard improvements can then present information produced by those services.

This ordering reduces rework because later features can reuse tested foundational components.

5. PLAN THE SPRINT BACKLOG

5.1 Sprint Planning Details

Sprint Number: Sprint 1
Sprint Duration: 2 weeks
Development Approach: Agile Scrum
Sprint Type: Foundational security and key management increment

Sprint 1 Goal: Establish the foundational satellite-node and cryptographic key management workflow by registering simulated nodes, verifying their identities, and generating keys through authorized requests, with important actions recorded in the audit log.
The first sprint concentrates on the essential backend workflow rather than attempting to complete the entire system. Contact-window distribution, interrupted-update recovery, detailed key rotation, and advanced monitoring will be developed in later sprints.

5.2 Selected User Stories

User Story 1 – Register a Satellite Node: 3 Story Points
As a ground control operator, I want to register a simulated satellite node with a unique identifier, so that the system can manage and monitor each satellite separately.
Reason for selection: Node registration provides the basic information needed by identity verification and key management.
User Story 2 – Verify Satellite Identity: 5 Story Points
As a cybersecurity administrator, I want the system to verify a satellite node's identity before allowing sensitive operations, so that unauthorized nodes cannot perform key management actions.
Reason for selection: Identity verification introduces an essential security control before sensitive key operations are permitted.
User Story 3 – Generate Cryptographic Keys: 5 Story Points
As an authorized key management operator, I want to generate cryptographic keys using an approved cryptographic library, so that authorized simulated satellite nodes can be provisioned with key material securely.
Reason for selection: Key generation demonstrates the central purpose of the project and builds on the registration and identity verification functions.
Total Sprint 1 Estimate: 13 Story Points
This is a provisional estimate based on an assumed small student development team and a two-week sprint. The team should confirm the workload during sprint planning and reduce the scope if its actual capacity, technical experience, or environment setup does not support completing all selected stories.

5.3 Development Tasks

Task 1: Analyze and Refine Requirements
- Review the acceptance criteria for node registration, identity verification, and key generation.
- Identify dependencies between the three stories.
- Define valid inputs, error cases, and expected outputs.

Task 2: Design the Satellite Node Data Model
- Define the fields required to identify a simulated satellite node.
- Establish validation rules for unique identifiers and required details.
- Define how node status will be represented.

Task 3: Develop Satellite Registration
- Implement the registration service and its validation rules.
- Prevent duplicate node identifiers.
- Return clear success and error responses.

Task 4: Design and Implement Identity Verification
- Define the identity verification workflow for registered simulated nodes.
- Implement the configured credential-validation process.
- Reject invalid or missing credentials before protected operations.
- Ensure authorization decisions are enforced by the backend.

Task 5: Implement Cryptographic Key Generation
- Configure the selected Python cryptographic library.

- Implement key generation using an appropriate algorithm and secure parameters.
- Associate the operation with the intended registered node.
- Keep raw key material out of ordinary database records, dashboard output, and audit logs.

Task 6: Implement Operation Records and Audit Logging

- Record registration, identity verification outcomes, and key-generation events.
- Include timestamps, event types, operation status, and relevant non-secret identifiers.
- Ensure failed operations are also recorded where appropriate.

Task 7: Integrate the Backend Workflow

- Connect registration, identity verification, and key generation services.
- Check that key generation cannot bypass required identity and authorization checks.
- Handle invalid requests and expected service errors.

Task 8: Test the Completed Stories

- Test successful and unsuccessful node registration.
- Test valid and invalid identity credentials.
- Test authorized and unauthorized key-generation requests.
- Verify that logs contain relevant metadata without exposing key material.
- Record defects and retest after corrections.

Task 9: Review and Demonstrate the Sprint Increment

- Compare completed functionality against the acceptance criteria.
- Demonstrate the registration-to-key-generation workflow.
- Record incomplete work and carry it into the product backlog.
- Collect stakeholder or faculty feedback for the next sprint.

5.4 Sprint Dependencies and Risks

Dependency 1: A node must be registered before the system can associate a key operation with it.
Dependency 2: Identity verification and backend authorization must be available before protected key generation can be accepted.
Dependency 3: Audit logging must be integrated with the operations being recorded.
Risk 1 – Cryptographic Integration: Incorrect configuration or handling of cryptographic material may create security weaknesses. The team will use an appropriate established cryptographic library and review its configuration.
Risk 2 – Scope Overload: Completing too many features in the first sprint may result in incomplete or poorly tested work. The team will prioritize the three selected stories and defer lower-priority functionality when necessary.
Risk 3 – Integration Defects: Individual services may work separately but fail when combined. Integration tests will be performed before the sprint review.
Risk 4 – Incomplete Audit Records: Some failed operations may not be recorded if logging is added too late. Audit requirements will be included in the implementation and testing of each relevant story.

5.5 Sprint 1 Deliverables

At the end of the proposed first sprint, the expected deliverables are:

1. A basic satellite-node registration service.
2. Validation that rejects duplicate or incomplete node registrations.
3. An identity verification workflow for registered simulated nodes.
4. A cryptographic key-generation service using the selected library.
5. Backend enforcement of authorization for protected key operations.
6. Audit records for important registration, authentication, and key-generation events.
7. Automated tests covering successful and failed workflows.
8. Technical documentation and a demonstration of the completed increment.

These are planned deliverables, not claims that the features have already been implemented. A deliverable will be accepted only after it meets its acceptance criteria and the required tests pass.

6. ESTIMATE EFFORT USING STORY POINTS

6.1 Estimation Technique

The project will use the **Fibonacci sequence** for relative story-point estimation: 1, 2, 3, 5, 8, and 13. Story points represent the relative complexity, effort, uncertainty, and integration risk of a user story. They are not direct measurements of hours or days.

The team can use Planning Poker during sprint planning. Each team member independently estimates a story, explains the reasoning, and discusses differences before agreeing on a team estimate.

6.2 Product Backlog Story-Point Estimates

User Story 1 – Register a Satellite Node: 3 points

The story requires input validation, unique identifier handling, persistence of node details, and appropriate responses. Its scope is relatively limited.

User Story 2 – Verify Satellite Identity: 5 points

The story requires identity validation, integration with protected operations, rejection of unauthorized requests, and tests for valid and invalid credentials. Security-related failure cases add complexity.

User Story 3 – Generate Cryptographic Keys: 5 points

The story requires cryptographic library integration, appropriate configuration, secure handling of key material, association with a node, and validation of operation results.

User Story 4 – Track Key Lifecycle and Rotation: 8 points

The story involves lifecycle states, rotation rules, operation status tracking, and handling of unsuccessful rotation attempts. It depends on the key-generation workflow.

User Story 5 – Distribute Keys During a Contact Window: 8 points

The story requires simulated communication-window state, encrypted transfer handling, completion conditions, and recording of transfer results.

User Story 6 – Recover from an Interrupted Key Update: 8 points

The story requires interruption handling, preservation of operation state, retry conditions, and prevention of inconsistent update results.

User Story 7 – Isolate a Suspicious Satellite Node: 8 points

The story requires configured detection conditions, access restrictions, status updates, audit records, and tests proving that isolated nodes cannot perform restricted operations.

User Story 8 – Maintain Audit Records: 5 points

The story requires structured event records, integration with multiple operations, protection of sensitive information, and verification of the tamper-evident record mechanism.

User Story 9 – Monitor the Satellite Fleet: 5 points

The story requires retrieving and presenting node status, key operation results, and security alerts while respecting access permissions.

User Story 10 – Manage Authorized User Access: 5 points

The story requires role and permission definitions, backend enforcement, rejection of unauthorized actions, and tests for access-control rules.

These estimates are initial planning values. They may change after technical investigation, testing, and team discussion reveal additional complexity.

6.3 Sprint 1 Estimation Summary

- Satellite-node registration: 3 points

- Satellite identity verification: 5 points
- Cryptographic key generation: 5 points

Total = 3 + 5 + 5 = 13 Story Points

The three stories form a coherent increment because registration establishes the node record, identity verification protects access, and key generation provides the main key management capability. Audit logging is integrated as a supporting task for these stories.

The estimate assumes that the development environment and selected libraries can be configured within the sprint. If environment setup or security review takes longer than expected, the team should preserve the security requirements and reduce the amount of functionality accepted into the sprint rather than declare unfinished work complete.

7. SPRINT GOAL AND EXPECTED DELIVERABLES

7.1 Sprint Goal

To deliver a tested foundational key management workflow in which an authorized operator can register a simulated satellite node, verify its identity, and request cryptographic key generation, while the system enforces access controls and records important operations without exposing raw key material.

This goal is specific, measurable through acceptance criteria, and aligned with the main purpose of the LEO Satellite Key Management Fabric.

7.2 Expected Sprint Deliverables

Deliverable 1: Satellite Registration

A working registration service that accepts valid node details, rejects duplicates, and records the node's status.

Deliverable 2: Identity Verification

A verification workflow that accepts valid configured credentials and rejects invalid or missing credentials before protected operations.

Deliverable 3: Cryptographic Key Generation

A service that generates key material through the configured cryptographic library and records the operation's result and associated metadata.

Deliverable 4: Audit Logging

Structured audit events for important operations, with sensitive key material excluded from ordinary logs.

Deliverable 5: Automated Test Results

Test evidence covering valid registrations, duplicate registrations, identity verification, access-control failures, key-generation requests, and relevant audit records.

Deliverable 6: Technical Documentation

Documentation describing the selected workflow, acceptance criteria, test cases, known limitations, and any incomplete tasks.

Deliverable 7: Sprint Demonstration

A demonstration showing the intended workflow from node registration to identity verification and authorized key generation. The demonstration must reflect the actual state of the prototype at the time of review.

7.3 Definition of Done

A user story will be considered complete only when:

1. Its acceptance criteria are satisfied.
2. The implementation is integrated with the relevant system components.

3. Required functional and security-related tests pass.
4. No unresolved critical defect prevents the feature from working as intended.
5. Sensitive key material is not exposed through ordinary database records, dashboard output, or audit logs.
6. Relevant audit events are recorded and can be reviewed.
7. The code and supporting documentation are reviewed and maintained in version control.
8. The completed functionality is demonstrated during the sprint review.

A story that has been partially developed but has not satisfied its acceptance criteria will remain incomplete and return to the product backlog.

7.4 Plan for Later Sprints

Sprint 2 – Secure Distribution and Contact-Window Simulation
Develop simulated encrypted key distribution, contact-window availability checks, and operation-status tracking.
Sprint 3 – Interrupted Updates and Key Rotation
Implement controlled retry handling, lifecycle-state management, and key rotation workflows.
Sprint 4 – Security Response and Monitoring
Develop simulated suspicious-node isolation, security alerts, and a more complete monitoring dashboard.
Sprint 5 – Integration, Security Testing, and Refinement
Perform end-to-end testing, review access controls, verify audit-log integrity, fix defects, improve documentation, and prepare the academic prototype demonstration.
These are provisional sprint themes. Their scope and estimates will be reviewed using feedback, actual progress, dependencies, and team capacity rather than treated as fixed commitments.

CONCLUSION

The Agile Sprint Planning and User Story Workshop provides a structured plan for developing the LEO Satellite Key Management Fabric through small, testable increments. The project vision, stakeholder roles, epics, user stories, and acceptance criteria connect the system's security and operational requirements to practical development activities.

The product backlog prioritizes foundational node management, identity verification, cryptographic key generation, and auditability before more complex features such as contact-window distribution, interrupted-update recovery, rotation, and compromised-node isolation. The first two-week sprint is provisionally estimated at 13 story points and focuses on a coherent, testable foundation.

Using Agile Scrum allows requirements to be reviewed and refined as the prototype develops. Acceptance criteria, automated tests, sprint reviews, and a clear Definition of Done provide evidence that completed features behave as expected. Security and reliability are treated as requirements throughout development rather than postponed until the final sprint.

The proposed plan supports the development of a controlled academic prototype. Its security and reliability must be evaluated through testing, and the prototype must not be represented as an operational satellite security system or as providing absolute protection against cyberattacks.